Can this benchmark measure context? Diagnostics for context-dependent code review

Tadael Shewarega Gebre
Independent researcher, Addis Ababa · tadaelshewaregagebre30@gmail.com

Preprint, 30 October 2026. Code, data and frozen results: github.com/iamTadaeltadi/codereview-copilot.

Abstract

Systems that give a language model more of the repository during code review change which code the model sees and how its relationships are described. Before asking whether either helps, we ask whether a benchmark can measure the help at all, and answer with four diagnostics and a record. A partial-input ceiling: under a prompt that presumes a defect, given only the diff, a model finds the mutated line in 93%, 97.8% and 95.5% of tasks on three successive versions of a generated cross-file benchmark, the last built after an external review had closed every known leak; the model flags most changed lines, so detection cannot separate one context condition from another. Under a neutral prompt that allows an empty answer the same diff-only rate on the third version is 30%, so the ceiling belongs to the prompt and the benchmark together. On 244 test-verified real defects the diff alone localises 51% at a five-line tolerance once the diff lines are numbered, 15 points above random line placement. An anchor-versus-evidence measurement on 291 real review comments: every one anchors on a changed file, yet 33 to 50% justify themselves with code the diff does not contain. A contrast set of safe twins, the same mutation placed where every caller in the repository tolerates it: under a prompt that presumes a defect no arm on two models leaves more than 11% of them unflagged; under a neutral prompt gpt-4o-mini leaves 82% alone from the diff, 54% when shown the caller that tolerates the change, 40% with a bare link stated between the snippets and 11% with the relation and its argument named, while foreign code of the same length changes nothing; deepseek-v3.2 moves in the same order by a tenth of the distance, from 89% to 79%. Under the same neutral prompt on the defects, the diff alone finds 30% of the mutated lines and the attributed link 82%, the mirror image of the twins, and balanced accuracy over defects and twins is 56% for the diff alone and no better for any arm that carries evidence; dependency metadata changes how often the model flags, not whether it can tell a defect from a safe change. In signal-detection terms the diff alone has d′ 0.37 and criterion +0.71; every arm that carries caller code or dependency metadata has d′ at or below 0.37 while the criterion falls to −1.07 with the relation named, so the context moves the threshold, not the sensitivity. On the stopped third-version run, showing the depended-on caller lowers precision by 14 points against the diff alone while foreign code of the same length does not, and stating the link between the snippets recovers none of it. A test-verification check: of 86 generated defects run against their repositories' own test suites, 17 are caught by an existing test. On the real defects, a graph-retrieval condition that appeared to localise 48% against 33 to 36% for the diff alone and for random or lexical retrieval turned out to have received the line span of the function containing the defect; with that span removed it localises 34%. Three pre-registered runs followed. With the retrieval repaired, every condition sat at the rate of random line placement, 36% against 36%, because the model was counting lines from hunk headers and mostly miscounting; numbering the diff lines raised every condition to about 50% and the exact-line rate from 3% to 24%, with no condition above the diff alone; and a final run with callers-first retrieval gives graph minus random-other-files +1.2 [−2.2, +5.0] on all 244 defects, and +12.5 [+5.0, +20.8] on a pre-registered subset of 64 defects whose reference comment cites code outside the diff, a subset that had been seen in exploratory analysis and is reported as a lead, not a result. A held-constant-evidence experiment on dependency metadata was designed, pre-registered and run twice; both runs are withdrawn, the second after external review found five instrument defects, and the ceiling that survived their repair is why it is not run a third time. Eight claims were withdrawn on the project's own evidence; everything, including the withdrawn runs and the controls that withdrew them, is released.

1. Introduction

A pull request changes one function. Three files away, another function relies on the behaviour that just changed. Nothing in the diff is wrong on its own terms, and a reviewer who reads only the diff cannot see the problem. Systems that give a language model more of the repository exist to close this gap, and the most elaborate of them build a dependency graph, retrieve the neighbourhood of the change, and present it to the model with its structure attached.

Two things change at once in that design. The model receives different code than it would from a plain lexical search, and it receives a statement of how the pieces relate. Published comparisons report the combined effect [1, 2]. They do not say which part is doing the work, because in an end-to-end comparison the retrieved set and its representation move together. We set out to separate them with a held-constant-evidence design: the same diff and the same snippets in every arm, and only the metadata that describes their relationships varied, from absent through scrambled, a bare link, a typed relation and a wrong pointer.

This paper reports what had to be true of the benchmark before that experiment could mean anything, and the finding that it was not. The partial-input problem is known from natural language inference [3, 4, 5]: if one part of the input already determines the answer, an experiment on the other part measures nothing. For code review the one-line version is to give the model the diff alone and see how often it finds the defect. On every generated benchmark we built it almost always did. The first version leaked through broken distractors and loose evidence; the second through line numbers in its labels, a resolver that accepted any same-name import, distractors that changed the syntax tree, and a defect that sat last in the diff six times in ten. An external review found those, we closed each one with a test, regenerated the benchmark from the same repositories, and under a prompt that presumes a defect the diff alone still found the defect line in 95.5% of tasks, because the model flags most changed lines and the one semantic change is always among them. Under a neutral prompt that allows an empty answer the same rate is 30%, so part of the ceiling was the instruction to flag. That is the result of this paper: a benchmark can pass every construction check and still be unable to measure context, the prompt and the harness can be the reason as much as the benchmark, and the checks that tell you are cheap enough to run first.

Three further diagnostics say where context could be measured and what else a benchmark needs. On 291 test-verified defects from a mined review benchmark [6], every reference comment anchors on a file the pull request changed, and a third to a half of them refer to code the diff does not contain, so where a comment sits is not where its justification lives; on those defects the diff alone localises a third, which leaves room. A contrast set [7, 8] of safe twins, the same mutation placed where every caller in the repository tolerates it, measures whether any context makes a model say a change is safe; under a neutral prompt it shows the opposite, that the caller which proves the change safe, and every statement about the dependency on top of it, makes the model flag it. And running each generated defect against its repository's own test suite says how many a test would have caught: a fifth of those that could be run.

On the one set with headroom we had a retrieval result: graph retrieval that reaches the depended-on caller appeared to localise 48% of the real defects against 33 to 36% for the diff alone and for random or lexical retrieval at the same budget. An external reviewer noticed that the graph condition's payload, alone among the conditions, carried the line span of the function containing the defect. A control that removes the span localises 34%, within noise of the diff. That result is withdrawn here, with the control beside it. Three pre-registered runs followed, each reported whatever it gave. The first, with the graph builder repaired and the seeds taken from the diff, put every condition at the rate of random line placement, 36% against 36%, and a check of the findings showed why: given a raw unified diff, the model placed fewer than one quoted line in ten on the line that holds it. The second numbered the diff lines, and every condition rose to about 50%, fifteen points above chance, with none above the diff alone. The third, with callers-first retrieval and a primary comparison fixed in advance, gives the graph +1.2 points [−2.2, +5.0] over random code from other files on all 244 defects, and +12.5 [+5.0, +20.8] on a pre-registered subset of 64 defects whose reference comment cites code outside the diff; that subset was chosen with the earlier runs in view, where it read +3.1 and −1.6, and we report it as a lead. A search-trigger test on a bare tool-

equipped model closes the frame: asked to review a diff it searched in none of forty tasks; told the defect depended on another file, it searched in all forty.

The contributions are the four diagnostics with their results, the generated benchmark with the generator, the audit and the twins that an experiment on context would need, the four-run record on real defects with the two harness faults it exposed, and the record of the study: eight claims made and withdrawn on the project's own evidence, the last three being the held-constant experiment on each generated benchmark and the retrieval result. Each measurement is stated with what it does and does not support.

2. Related work

2.1 Context for LLM code review

AACR-Bench [2] evaluates automated review on 1,505 issues and reports that context granularity, with arms for diff, file and repository, and the choice of retrieval method both change performance, with effects that vary by model and language. It labels 233 of its 1,505 issues as repository-level, about 15%, so most of its ground truth is resolvable from the changed file alone. c-CRAB [6] converts human review comments into tests that fail before the fix and pass after it; its release carries 485 reference comments on 339 Python pull requests at the test-verified stage, of which 291 meet the filters in 3.6; it evaluates finished reviewers rather than context strategies, and we reuse its defects for a secondary measurement of anchor scope against evidence scope. SWR-Bench [9], CodeFuse-CR-Bench [10] and MCR-Bench [11] broaden the task to whole pull requests and multi-round review; MCR-Bench includes a long-range dependency miss error category, which names the failure we study without isolating its cause. SWE-PRBench [12] evaluates 350 pull requests and finds that all eight frontier models degrade monotonically as same-file context is appended to the diff, which its authors call attention dilution; its Type3_Latent class, 26% of defects, resides in files that import the changed files. Together these benchmarks establish that cross-file defects exist, that more raw context does not reliably help, and that retrieval choice matters, but in each the content of the context and its representation move together. This paper adds the check that has to precede any such comparison, a partial-input ceiling measured on three versions of a generated cross-file benchmark whose evidence is verified on the caller's AST, together with an anchor-versus-evidence measurement on the 291 c-CRAB defects and four controlled retrieval runs on them in which no selection beats the diff on the full defect set.

2.2 Graph-structured retrieval for code

RepoGraph [13] builds a repository-level graph of definitions and references and retrieves ego-graphs around matched entities for agents; our graph construction follows it. GRACE [1] couples graph-guided retrieval with structural fusion of the retrieved subgraph into the prompt and reports a gain of 8.19 exact match on repository-level completion; because the retrieved set and its structural encoding change in the same step, the gain cannot be attributed to either alone. DyRetriever [14] and LARGER [15] report further gains from graph-guided methods on generation and localisation, which shows that graphs help when the bottleneck is finding the right code. GrepRAG [16] reports that ripgrep over the repository matches or beats graph retrieval on CrossCodeEval, and Agent Retrieval Bench [17] finds that no retriever dominates, although any retrieved seed beats a random non-gold seed. These results bound the retrieval question on both sides: a graph can select better evidence, and a cheap lexical search can often select it just as well. None of them separates the selection of evidence from the statement of how the selected pieces relate. This paper reports a selection comparison on real review defects with random-code and lexical controls at a fixed budget, in which graph retrieval appeared to win by fifteen points until a span it alone had received was removed, and releases the held-constant design that would separate selection from description once a benchmark can carry it.

2.3 Do LLMs use serialised structure?

Outside code, the evidence that prompted models read graph structure is mixed. On text-attributed graphs, "When Structure Doesn't Help" [18] finds node text alone strong and structural encodings marginal or negative. "Actions Speak Louder than Prompts" [19] uses shuffled-adjacency controls and finds that prompting-mode models lean on node features while a code-execution mode does use the structure; our scrambled control follows the same logic. "Lost in Serialization" [20] shows that graph reasoning is not invariant to node labelling or encoding, and [21] traces part of this to RoPE-induced attention decay between nodes that are adjacent in the graph but distant in the linearisation. GraphSOS [22] and GraphDO [23] show that serialisation order alone moves accuracy, with GraphDO reporting 89.43% under BFS order against 78.36% under random order. For code, "Representation Matters" [24] reports that prompts built from AST and PDG views beat raw source on vulnerability reasoning, 83.2 against 53.5; there the structural view replaces the source, so the comparison measures a change of content as much as a change of representation. Our arms add metadata to constant content, so the two designs answer different questions, and we read that result as a ceiling on what structure can carry rather than as a measurement of its marginal value. Closest to our twins, "Words Speak Louder Than Code" [25] holds the code fixed and varies only its surroundings, an author attribution, a framing, a prior analysis, and finds that all eight models it tests change verdicts, often from safe to vulnerable without identifying a vulnerability; our twins ask the same question of structural dependency context, the caller and the statement of the relation, and find the same pattern. This paper's held-constant design was built to ask the same question for review, with two serialisations, a comprehension probe and validated judges; its two runs are withdrawn with the benchmarks they ran on, and what is reported is the diagnostic that gates it.

2.4 Context length and distractors

"Context Length Alone Hurts" [26] shows that padding a prompt with irrelevant text degrades performance independently of the task. RepoReasoner [27] finds that longer context is not consistently better and that models struggle on cross-file reasoning even with oracle context, so perfect evidence does not guarantee the right inference. "When Retrieval Hurts Code Completion" [28] identifies cases in which retrieved context lowers accuracy. Two review-specific results pull in different directions. "Measuring and Exploiting Contextual Bias in LLM-Assisted Security Code Review" [29] finds that non-code metadata can override the semantic content of the code, which is the mechanism by which corrupted dependency metadata could cause harm. "Can Adversarial Code Comments Fool AI Security Reviewers" [30] finds that adversarial comments did not reduce detection. Neither study varies structural metadata about the code under review while holding the code fixed. This paper adds, on real defects, random-code controls at the same budget drawn from the same repository and from other files only, and a control that strips a location hint from the one condition that carried it, so that reaching the dependent code can be told apart from adding tokens and from being told where to look; and it reports that cosmetic distractors in a generated diff do not prevent a model from finding the one semantic change.

2.5 Agents, exploration, and mined regressions

Whether an agent looks for a dependency on its own depends on the harness. CodeCompass [31] reports that claude-sonnet-4-5 made zero tool calls in 58% of trials despite an instruction to search, and that a trailing checklist raised this to 100%. "Agentic Code Review in the Terminal" [32] finds that harnessed products such as Claude Code and Gemini CLI take 5.5 to 7.5 exploration steps unprompted, and OpenCodeReview [33] offers an open implementation of such a reviewer. Any claim that agents do not search therefore has to be scoped to a bare tool-equipped model. For ground truth, CR-Bench [34] mines 584 bug-inducing pull requests from SWE-bench fixes with an SZZ-style procedure, and Song et al. [35] characterise 280 Java regressions, including compatibility errors in which a correct change violates an assumption held elsewhere in the repository; AgentSZZ [36] automates the mining step with an agent. These are the prior art for mined cross-file regressions and the natural external check on a generated benchmark. This paper adds a search-trigger test on a bare tool-equipped model, scoped as above, and relates its generated cross-file defects to the mined compatibility class rather than replacing it.

2.6 Framing, pairs, and judges

Three results from 2026 bear directly on the twin finding and the judge. "Measuring and Exploiting Contextual Bias in LLM-Assisted Security Code Review" [29] shows, on 250 CVE and patch pairs across six models, that a natural-language assertion that code is safe collapses detection, from 97% to 4% for gpt-4o-mini, while an assertion that it is buggy barely moves it. Our twins vary the caller shown, not text about safety, and Section 4.3 reports whether a tolerant caller lowers the model's willingness to flag. "Are LLMs Reliable Code Reviewers?" [37] finds that asking a model to explain raises its misjudgement of correct code, which bears on any twin result under a prompt that presumes a defect. Matched-pair designs in modernisation [38] and vulnerability detection [39] score a pair correct only when both members are, and report pairwise accuracies near 0.1; our twins are the caller-context version of that design, verified on the syntax tree, and we report the twin half separately because the pairs are matched by mutation kind rather than by function. "Bias in the Loop" [40] shows that model judges flip on presentation cues with code unchanged; the results reported here use no model judge. This paper adds a twin benchmark whose safe member is safe by construction, checked across every file of the sampled repository rather than by annotation.

3. Method

The question we set out to answer is whether telling a model how retrieved code is connected helps it review a change, once the code itself is already in front of it. Prior comparisons cannot answer this because retrieval and representation change together. The design that can, holding the snippets constant and intervening only on the metadata that describes their relationships, needs a benchmark on which the diff alone does not already give the answer. This section describes the benchmark, the twins, the four diagnostics that test whether the benchmark can carry the experiment, and the real-defect set on which a retrieval result is reported. The held-constant design itself is described briefly in 3.5, because its two runs are withdrawn and the diagnostics are the reason.

3.1 Task

A reviewer receives a pull-request diff and must report defects as a list of `(file, line, message)`. Each task contains exactly one defect, introduced by a mutation in one file, that is wrong only because of an assumption a different file makes. The diff also contains three harmless edits, so the reviewer must say *which* change is wrong rather than *that* something is.

3.2 Benchmark construction

Tasks are generated from 39 production repositories at a fixed commit, each sampled to at most 300 production files. For every public function with a caller in another file, the generator attempts one of five mutations and keeps the task only when the caller's syntax tree demonstrates the dependency the mutation breaks:

Kind	Mutation	What the caller must contain
`exception_type`	`raise A` → `raise B`	a handler that catches `A` and not `B`, enclosing the call
`none_sentinel`	`return None` → `return -1`	a comparison of the call's result with `None`
`default_flip`	`p=True` → `p=False` in the signature	a call that omits `p`, positionally and by keyword
`tuple_order`	`return a, b` → `return b, a`	an unpacking assignment of the call's result
`empty_to_none`	`return []` → `return None`	iteration over, `list()` of, or `len` of the call's result

Calls are resolved through imports: a bare name imported from the defining module, or re-exported from it through package `__init__` files followed to three levels, or `module.name` after a module import. A same-name import from any other module is not a call to the mutated function. Attribute calls on arbitrary objects are not matched, which excludes most methods and removes name collisions between unrelated functions called `run` or

`write`. Each task records the verified caller line and the *argument* of the dependency: the exception type, the parameter name, the unpacked names.

Distractors are rewrites that leave the file's syntax tree identical: a change of string quote style in either direction, redundant parentheses around a return value or a right-hand side, a spacing change. Each is applied to a copy of the whole file and rejected unless the tree is unchanged. Three are chosen per task from a window around the defect whose centre is jittered and whose split above and below the defect is seeded, so the defect's position among the four hunks is close to uniform: in the third version it is last in 28% of diffs. The diff lists the four hunks in line order with nothing marking which is which. Where the generator names a snippet, in the metadata blocks of 3.5, it uses the file stem and the enclosing function, never a line number, because the diff shows line numbers too.

The third version holds 277 defects from 37 repositories: 144 default flips, 91 tuple reorderings, 27 `None` sentinels, 8 exception types and 7 empty-to-`None`. An audit script re-checks every released task with the generator's own resolver and tree comparison, following re-exports through the cached repository: 0 of 277 have a caller import that does not resolve to the mutated module, 0 have a distractor that changes the tree.

Two earlier versions are withdrawn. The first located evidence by a regular expression within eight lines of any same-named call, which demonstrated the dependency in roughly a third of tasks, and its distractor rewriter broke docstrings in 145 of 204 tasks, so a model that flagged a syntax error was scored as a false positive. The second was audited after an external review on 26 October 2026 and had five defects: its metadata labels carried the mutated line's number, which the diff also shows; its resolver accepted a same-name import from any module, in 36 of 229 tasks; 25 of its 150 twins had a dependent call in one of the two files shown; 91 of its 650 distractors changed the syntax tree, 639 of the 650 were quote rewrites, and the defect sat last in the diff in 62% of tasks; and an unparseable reply was scored as correctly silent. Each has a pinned test in the artifact. The present generator was written against both lists.

3.3 Safe twins

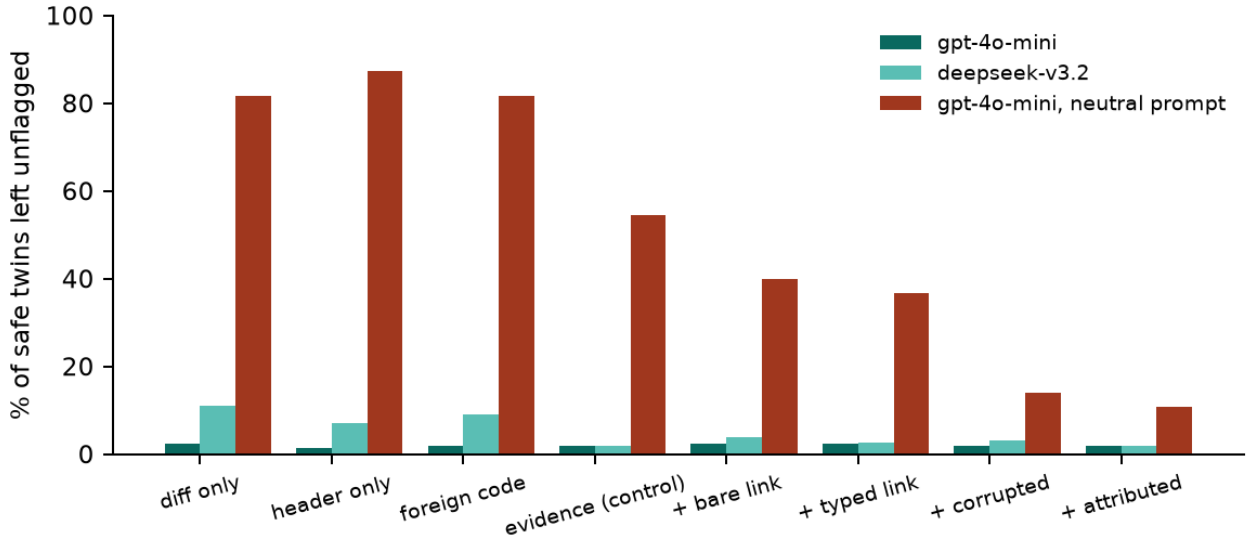


Figure 3. Safe twins on the third benchmark: the share of changes every sampled caller tolerates that the model leaves unflagged, by arm, on two models under the presuming prompt and on gpt-4o-mini under the neutral prompt.

A twin is the same surface mutation as a defect task placed where no caller depends on what changed: a flipped default that every call site passes explicitly, a changed exception type that no enclosing handler catches. Safety is checked repository-wide, not on the two files shown: every non-test file in the pool is scanned for a bound call to the function, or an attribute call of the same name, that the kind's verifier would accept, and the mutation is

rejected as a twin if any is found. The third version holds 165 twins from 33 repositories, 149 default flips and 16 exception types; the audit finds 0 with a dependent call. A twin task is rendered like a defect task, with the same distractors and, in the evidence arms, a caller window that shows a call the change does not break. Correct on a twin means not flagging the mutated line.

3.4 Four diagnostics

Partial-input ceiling. The model is given the diff alone and must report defects; a hit is a finding whose line equals the mutated line, with no tolerance. The rate is reported for every version of the benchmark under the presuming prompt of 3.6, for the third version under the neutral prompt as well, and for the real-defect set. A rule for the third version was fixed before its run: if the diff-only hit rate was 95% or higher, the remaining arms would not be run and only the ceiling would be reported; the target was under about 80%.

Anchor versus evidence. For each real reference comment, `experiments/evidence_scope.py` records where the comment is anchored (a changed file, a changed line) and whether its text refers to anything the diff does not contain, under a strict reading (a named identifier, file or behaviour absent from the diff) and a loose one (any reference beyond the changed lines). Every decision carries its reason.

Safe twins. As in 3.3; the clear rate per arm is the diagnostic, compared between the diff-only arm and the arm that shows the tolerant caller.

Test verification. `experiments/verify_by_tests.py` clones each repository at the benchmark commit, installs it, selects the tests nearest the mutated module, runs them before the mutation, applies the mutation and runs them again. A defect is test-verified when a test that passed before fails after. The funnel, including repositories that could not be installed and modules with no passing test, is reported in full.

3.5 The experiment the diagnostics gate

The held-constant-evidence design fixes an evidence set per task, a 25-line window around the mutated line, a 25-line window around the verified caller line and a decoy function from the definition file, and renders it identically in every arm that receives it. Arms differ only in a metadata block: absent; scrambled so that it has the same shape and says nothing true; a bare link; a typed relation; a typed relation with its argument; or a relation with its target moved to the decoy. Two further arms replace the snippets with length-matched functions from repositories outside the benchmark, or remove them while keeping the metadata section. The primary comparison, bare link against scrambled control on a metric that asks whether the message explains the defect through the caller, was recorded in the repository (commit 71fb660, 1 October 2026) before the second version's runs. Both versions' runs are withdrawn with the benchmarks they were run on. A third run was conditional on the diagnostic in 3.4, and Section 4.1 reports why it was stopped after one arm set. The runner, arms, serialisations, judges and analysis remain in the artifact for a benchmark that passes the gate.

3.6 The real-defect set, and the reviewer prompt

c-CRAB's paper reports 184 pull requests and 234 comments; its public release holds 410 instances and 595 comments, and its stage-three test-verified file holds 339 Python pull requests with 485 reference comments. We use that file and the test-generation archive, downloaded on 27 August 2026, and filter in four steps: 477 comments carry a line; 340 of those carry a generated test that fails before the fix and passes after it; 291 of those sit in a source file of a supported language, and these are the set for the anchor-versus-evidence measurement; 246 remain after diffs longer than 40,000 characters are dropped (195 pull requests). Section 4.5 reports four runs on these 246 defects; each is analysed on the defects answered in all of its conditions, 219 to 223 in the first run and 239, 243 and 244 in the three that followed, the rows missing being replies the runner could not score or, in the last run, one task whose sources could not be fetched.

The system prompt is one text for every arm and both benchmarks. It tells the model that humans have reviewed the diff and found real problems, to assume there is something to find, and the output contract asks for best

candidates even when uncertain while allowing an empty list. The sentence was written for the real-defect runs, where every diff carries a confirmed defect, and was kept for the generated benchmark and the twins so that arms differ only in the metadata. It pushes the model toward flagging, so the twin clear rates in 4.3 are a floor on what a neutral prompt would give. A neutral variant that drops the sentence and states that an empty list is a correct answer is implemented (`--neutral-prompt`); it is the prompt of the neutral runs in 4.3.

3.7 Models and statistics

The ceiling and the test-verification check use gpt-4o-mini; the twins use gpt-4o-mini and deepseek-v3.2; the search-trigger test uses gemini-2.5-flash; all at temperature 0.1 through one API. Tasks are paired across arms. The reported interval is a 95% percentile bootstrap that resamples repositories rather than tasks, because several tasks share a repository and a definition file, with the task-resampled interval shown beside it in the analysis files. McNemar's exact test on discordant pairs is reported for each comparison. "Excludes zero" and "spans zero" are the only verdicts used, and a result is reported only from a run that completed on its full paired set.

4. Results

All numbers come from the files under `results/`, recomputed from the frozen per-call records by `scripts/reproduce.py`. Intervals are 95% percentile bootstraps that resample repositories; the task-resampled interval, shown in the analysis files, never changes a verdict. Nothing is reported from a run that did not complete on its paired set; the one stopped run is described in 4.7 and kept whole.

4.1 The partial-input ceiling

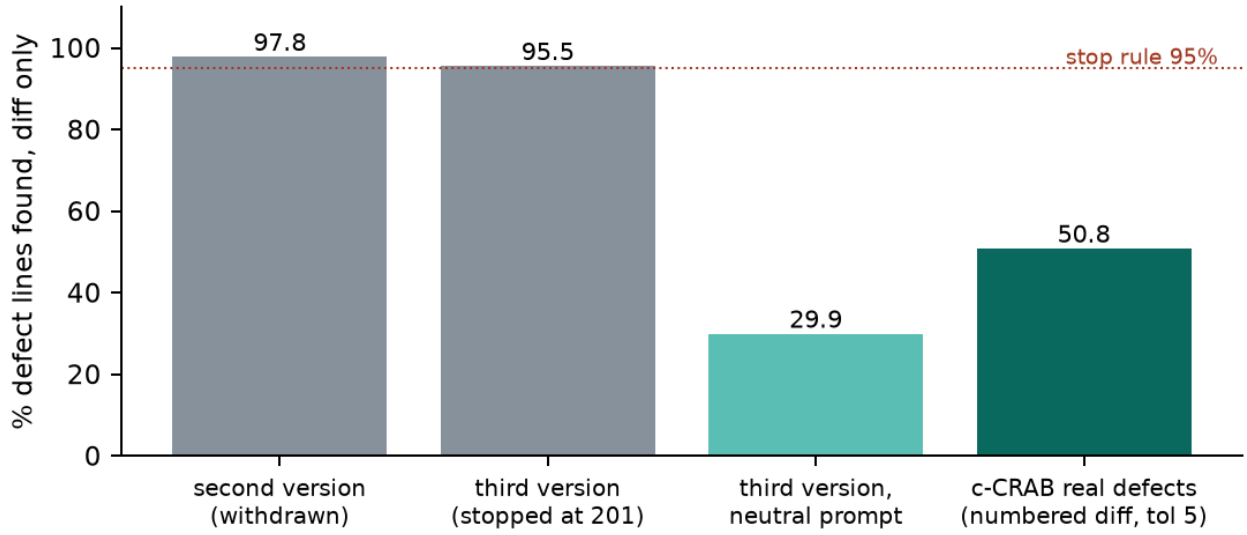


Figure 1. The partial-input ceiling: share of defect lines gpt-4o-mini finds from the diff alone, at the exact line, on the second and third generated benchmarks under the presuming prompt, on the third under the neutral prompt, and on the real c-CRAB defects (numbered diff, 5-line tolerance, 244 defects; random placement gives 35%). The third version was stopped under the pre-set 95% rule.

Under the presuming prompt of 3.6, a model given nothing but the diff finds the mutated line almost always, on every version of the generated benchmark (gpt-4o-mini, a hit is the exact line):

Benchmark version	tasks	diff-only hit	what was wrong with the instrument
first (withdrawn)	204	93%	regex evidence; distractors broke the file in 145 tasks

second (withdrawn)	229	97.8%	line numbers in labels; loose resolver; tree-changing distractors; defect last in 62%
third	201 of 277	95.5%	none found: audit 0 / 0 / 0, position balanced, labels name functions
third, neutral prompt (4.3)	271	29.9%	the prompt no longer says a defect is present
c-CRAB real defects (3.6), raw diff, 5-line tolerance	219	32.9%	the rate of random line placement (4.5)
c-CRAB real defects, numbered diff, 5-line tolerance	244	50.8%	15 points above random placement

The third version was run under the stop rule of 3.4 and stopped at 201 defects when the rate was clearly above 95%; there were no parse failures in 1,802 replies. By kind the rate is 104 of 107 default flips, 58 of 61 tuple reorderings, 19 of 21 `None` sentinels, 6 of 6 exception types and 5 of 6 empty-to-`None`. The reason is not that the model reads the defect: it reports 2.66 findings per task on average against four hunks, and 1.58 of those findings are on the three harmless rewrites. A semantic change among three cosmetic ones is always among the lines it flags. Precision, the rate at which the mutated line is the only line flagged, is 24.4%.

Repairing the second version's leaks changed the ceiling by two points. Balanced position, function-name labels, mixed syntax-tree-neutral distractors and a strict resolver remove the ways a model could find the line without reading the change; they do not remove the fact that a flipped default or a swapped return among quote rewrites looks like the only thing worth saying. Detection rate on a mutation benchmark of this shape measures whether the mutated line looks odd, and it does. Every generated-benchmark rate in the table is under a prompt that tells the model a defect is present. Under the neutral prompt the third version's diff-only rate is 29.9% (4.3): the ceiling is a property of the benchmark and the prompt together, and once the prompt no longer presumes a defect the arms separate, by flagging rather than by discriminating, as 4.3 shows. The same check on the real defects gives 32.9% on the raw diff at a five-line tolerance, which 4.5 shows to be the rate of random line placement, and 50.8% once the diff lines carry their numbers. That set leaves room for context; 4.5 reports four runs on it.

4.2 Anchor versus evidence, on real comments

Of 291 test-verified c-CRAB defects, 291 anchor in a file the pull request changed and 213 on a changed line. Under the strict reading, 97 (33%) refer to something the diff does not contain; under the loose reading, 146 (50%). Where a comment sits is not where its justification lives. A benchmark that scores a review by where its comment lands will accept a comment that found the line for the wrong reason, and a benchmark that gives the model only the diff withholds the justification for a third to a half of its own ground truth.

4.3 Safe twins

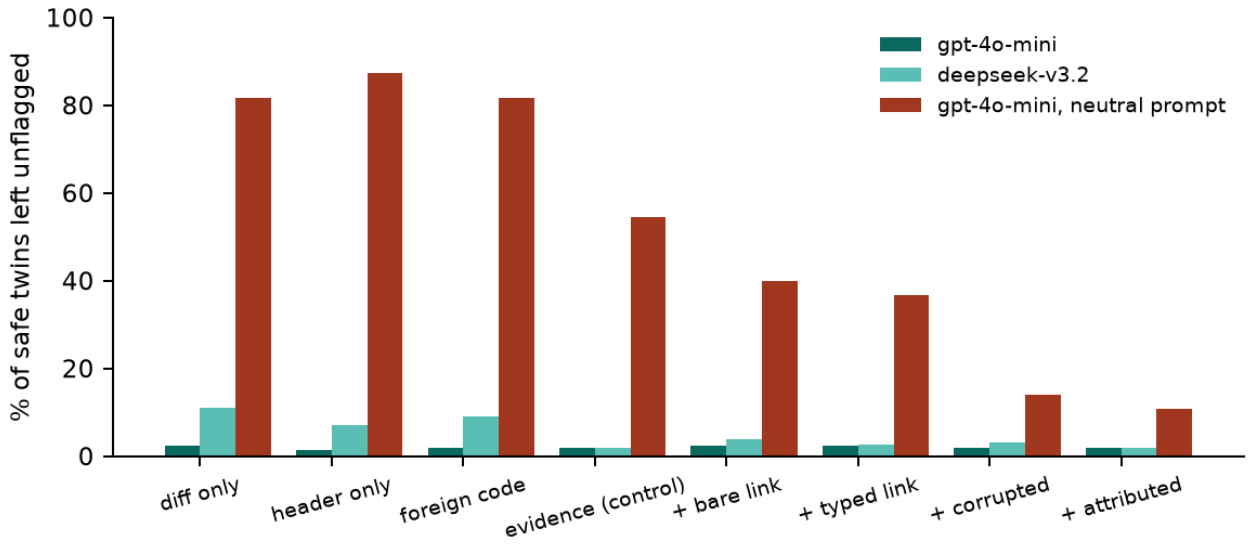


Figure 3. Safe twins on the third benchmark: the share of changes every sampled caller tolerates that the model leaves unflagged, by arm, on two models under the presuming prompt and on gpt-4o-mini under the neutral prompt.

Twins are the same surface mutation as a defect task, placed where no caller in the sampled repository depends on what changed (3.3). The generator produced 165 from 33 repositories; 158 have an evidence set and were run on both models, and 157 (gpt-4o-mini) and 155 (deepseek-v3.2) are paired across all nine arms. Correct on a twin means not flagging the mutated line. Both runs used the system prompt of 3.6, which tells the model that human reviewers found real problems in the diff and to assume there is something to find; every clear rate below is therefore a floor, and a neutral-prompt run that drops that sentence and states that an empty list is a correct answer is reported at the end of this section (Figure 3).

Arm	gpt-4o-mini (n=157)	deepseek-v3.2 (n=155)
1 diff only	2.5%	11.0%
8 header only	1.3%	7.1%
7 foreign code, same length	1.9%	9.0%
2 tolerant caller shown	1.9%	1.9%
3 caller + bare link	2.5%	3.9%
4 caller + typed link	2.5%	2.6%
5 caller + attributed link	1.9%	1.9%
6 caller + wrong link	1.9%	3.2%

Comparison on twin clear rate	gpt-4o-mini	deepseek-v3.2
tolerant caller − diff only	−0.6 [−3.0, +1.4]	**−9.0 [−14.1, −4.8]**
caller + attributed − diff only	−0.6 [−3.0, +1.4]	**−9.0 [−14.3, −4.2]**
foreign code − diff only	−0.6 [−3.0, +1.5]	−1.9 [−7.6, +3.2]
header only − diff only	−1.3 [−3.8, +1.2]	−3.9 [−9.8, +1.8]
bare link − caller alone	+0.6 [0.0, +2.1]	+1.9 [−0.7, +4.9]

No arm on either model leaves more than 11% of the safe changes unflagged. On gpt-4o-mini every arm sits between 1% and 4%, a floor under which no comparison can be read. On deepseek-v3.2 the diff alone clears 11%; showing the caller that tolerates the change lowers that to 2%, by nine points with an interval that excludes zero,

while foreign code of the same length and the header alone leave it within noise of the diff. Stating the link, in any form, moves nothing. The code that should exonerate the change makes the model more certain it is a defect, which is the same direction as the precision signal in 4.7: a relevant caller raises flagging, and a relation statement does not lower it. The twins on this version are safe repository-wide, so the second version's objection, that a caller elsewhere might not be robust, no longer applies within the sampled files.

Under a neutral prompt. The same 158 twins were re-run on both models with the one sentence removed and the output contract stating that an empty list is a correct answer (3.6); every arm is otherwise identical, and there were no parse failures in 1,422 replies on either model (`results/v3twins-neutral-gpt4omini/`, `results/v3twins-neutral-deepseek/`; the deepseek run was made as three interleaved shards of the same file in parallel, with identical per-call settings, because that model reasons three to four times longer under this prompt).

Arm, neutral prompt	gpt-4o-mini left unflagged (n=158)	findings per twin	deepseek-v3.2 left unflagged (n=158)	findings per twin
1 diff only	81.6%	0.19	89.2%	0.11
8 header only	87.3%	0.14	88.0%	0.14
7 foreign code, same length	81.6%	0.20	90.5%	0.09
2 tolerant caller shown	54.4%	0.50	86.7%	0.14
3 caller + bare link	39.9%	0.64	81.0%	0.20
4 caller + typed link	36.7%	0.69	77.2%	0.24
6 caller + wrong link	13.9%	0.96	82.3%	0.20
5 caller + attributed link	10.8%	0.97	79.1%	0.23

Comparison, neutral prompt (repositories resampled)	gpt-4o-mini	deepseek-v3.2
tolerant caller − diff only	**−27.2 [−37.7, −16.7]**	−2.5 [−8.5, +3.7]
foreign code − diff only	0.0 [−4.4, +4.8]	+1.3 [−4.4, +7.2]
header only − diff only	+5.7 [0.0, +11.6]	−1.3 [−5.9, +4.1]
bare link − caller alone	**−14.6 [−20.8, −8.4]**	**−5.7 [−11.3, −0.7]**
wrong link − caller alone	**−40.5 [−49.1, −31.2]**	−4.4 [−10.3, +1.6]
attributed link − diff only	**−70.9 [−77.5, −63.6]**	**−10.1 [−18.1, −2.1]**

Told that an empty answer is acceptable, gpt-4o-mini leaves four in five safe changes alone from the diff, and foreign code of the same length changes nothing. Showing the caller that tolerates the change costs 27 points; stating a bare link between the snippets costs 15 more; naming the relation and its argument takes the clear rate to one in nine, and a link that points to the wrong snippet does nearly the same as the right one. Every unit of dependency metadata moves the model toward flagging a change the metadata itself shows to be safe, in proportion to how much the metadata says. deepseek-v3.2 moves in the same order but by a tenth of the distance: 89% cleared from the diff, 87% with the caller, 81% with the bare link, 79% with the relation named, and only the attributed link against the diff and the bare link against the caller exclude zero. The direction of the effect is shared by the two models; its size is a property of the model. The presuming prompt had hidden all of this at a floor of 2.5%; under it the arms were indistinguishable, and the two-point fall on deepseek was the visible edge of an effect that is twenty-seven points on one model and two on the other.

The defects under the same prompt, and balanced accuracy. The 277 third-version defects were run on gpt-4o-mini under the neutral prompt with the same nine arms (`results/v3-neutral-gpt4omini/`; 6 defects had no evidence set, 271 are paired, 0 parse failures in 2,439 replies). The diff alone finds 30% of the mutated lines, not 95%: the ceiling of 4.1 was measured under a prompt that told the model a defect was there, and without that sentence the generated benchmark has headroom. Every unit of metadata fills it, in the same order as it empties the twins.

Arm, neutral prompt, gpt-4o-mini	defects hit (n=271)	exact line	findings per defect	twins cleared (n=158)	balanced accuracy
1 diff only	29.9%	29.5%	0.36	81.6%	55.8%
8 header only	32.8%	31.4%	0.39	87.3%	60.1%
7 foreign code, same length	34.7%	32.8%	0.45	81.6%	58.2%
2 tolerant caller shown	52.4%	50.9%	0.65	54.4%	53.4%
3 caller + bare link	63.1%	61.6%	0.80	39.9%	51.5%
4 caller + typed link	64.9%	61.3%	0.85	36.7%	50.8%
6 caller + wrong link	78.2%	75.6%	0.96	13.9%	46.1%
5 caller + attributed link	81.5%	77.9%	1.05	10.8%	46.2%

Comparison, neutral prompt, gpt-4o-mini	defects hit	balanced accuracy
tolerant caller − diff only	**+22.5 [+14.9, +30.5]**	−2.4 [−9.7, +5.1]
foreign code − diff only	+4.8 [+0.4, +9.0]	+2.4 [−1.0, +5.8]
header only − diff only	+3.0 [−1.1, +7.0]	+4.3 [+0.9, +7.8]
bare link − caller alone	**+10.7 [+5.1, +16.3]**	−1.9 [−6.3, +2.1]
wrong link − caller alone	**+25.8 [+19.7, +32.0]**	**−7.3 [−12.5, −2.3]**
attributed link − diff only	**+51.7 [+43.9, +59.3]**	**−9.6 [−14.9, −3.8]**

Balanced accuracy is the mean of the hit rate on defects and the clear rate on twins; defects and twins are different tasks, so the bootstrap pairs them by repository, drawing repositories with replacement and scoring both arms and both halves on the same draw (`analysis-balanced-with-twins.txt`). Each arm's gain on the defects is matched by its loss on the safe changes, almost point for point: the caller gains 22 and loses 27, the attributed link gains 52 and loses 71, and a link to the wrong snippet raises detection by 26 points over the caller alone, nearly as much as the right one. Balanced accuracy is 55.8% for the diff alone, within noise of that for the caller, for foreign code and for the bare link, four points above it for the header alone, and ten points below it for the attributed link. No arm that carries evidence beats the diff. Under a prompt that lets the model say nothing, the generated benchmark measures how much the context makes the model flag; it does not show the context helping the model tell a defect from a safe change. The defects were not run on deepseek-v3.2 under the neutral prompt.

Sensitivity and criterion. The same two rates read as one detection task give a sensitivity and a decision threshold: with H the hit rate on defects and F the flag rate on twins, $d' = z(H) - z(F)$ and $c = -(z(H) + z(F))/2$, repository bootstrap as above (`experiments/signal_detection.py`, `analysis-signal-detection.txt`).

Arm, neutral prompt, gpt-4o-mini	H	F	d′ [95%]	c [95%]
1 diff only	0.30	0.18	0.37 [+0.05, +0.70]	+0.71 [+0.60, +0.85]
8 header only	0.33	0.13	0.70 [+0.42, +1.00]	+0.79 [+0.64, +0.98]
7 foreign code, same length	0.35	0.18	0.51 [+0.25, +0.79]	+0.65 [+0.51, +0.80]

2 tolerant caller shown	0.52	0.46	0.17 [−0.12, +0.46]	+0.03 [−0.12, +0.16]
3 caller + bare link	0.63	0.60	0.08 [−0.20, +0.34]	−0.30 [−0.46, −0.14]
4 caller + typed link	0.65	0.63	0.04 [−0.28, +0.36]	−0.36 [−0.50, −0.23]
6 caller + wrong link	0.78	0.86	−0.30 [−0.59, −0.04]	−0.93 [−1.11, −0.77]
5 caller + attributed link	0.82	0.89	−0.34 [−0.67, −0.05]	−1.07 [−1.26, −0.91]

Every arm that carries the caller's code or a statement about the dependency has d′ at or below the diff's 0.37, and for the attributed and wrong links d′ is below zero: the model flags the safe twin more often than the defect. The criterion moves from +0.71, a model that needs persuading, to −1.07, one that flags by default; the difference in c from the diff excludes zero for every evidence arm (−0.69 [−0.85, −0.54] for the caller alone, −1.78 [−2.00, −1.60] for the attributed link), while the difference in d′ excludes zero only for the attributed and wrong links, downward (−0.72 [−1.13, −0.29] and −0.68 [−1.09, −0.28]). The only arm that raises d′ by an interval excluding zero carries no evidence at all, the header alone (+0.32 [+0.06, +0.59]). Dependency context in this experiment moves the threshold, not the sensitivity.

4.4 Test verification of the generated defects

Of the second version's defects, 86 were run against their repositories' own test suites (`results/test-verified/`): 17 are test-verified, meaning a test that passed before the mutation fails after it; 28 are not caught by the tests selected for the mutated module; 24 repositories could not be installed in the sandbox (native builds, a Python 3.13 requirement); 11 had no passing test before the mutation and 6 had no test to select. Among the 45 that could be run to a verdict, 38% are caught by an existing test. The test-verified defects are 9 default flips, 6 tuple reorderings and 2 `None` sentinels, from eight repositories. The same check on all 277 third-version defects (`results/test-verified/summary-v3.txt`) reaches a verdict for 59: 32 are test-verified and 27 are not caught, so 54% of those with a verdict, and 12% of all 277, are caught by an existing test. The rest could not be run: 159 repositories failed to install in the sandbox, the third version's pools include heavier projects than the second's, 46 modules had no passing test before the mutation, 7 had no test to select, and 6 timed out or errored. The test-verified defects are 16 default flips, 15 tuple reorderings and one `None` sentinel, from nine repositories; their task identifiers are listed in the summary file, so a reader who wants the executable subset has it.

The syntax-tree verifier accepts many mutations whose breakage no existing test observes. That does not make them harmless: a flipped default on a public function that no test pins is the kind of change a reviewer exists to catch. It does mean that a reader who counts only test-verified defects should treat the generated benchmark as resting on an unverified majority, and that a benchmark of this kind should report the fraction.

4.5 Real defects: four runs, one withdrawal, and what stands

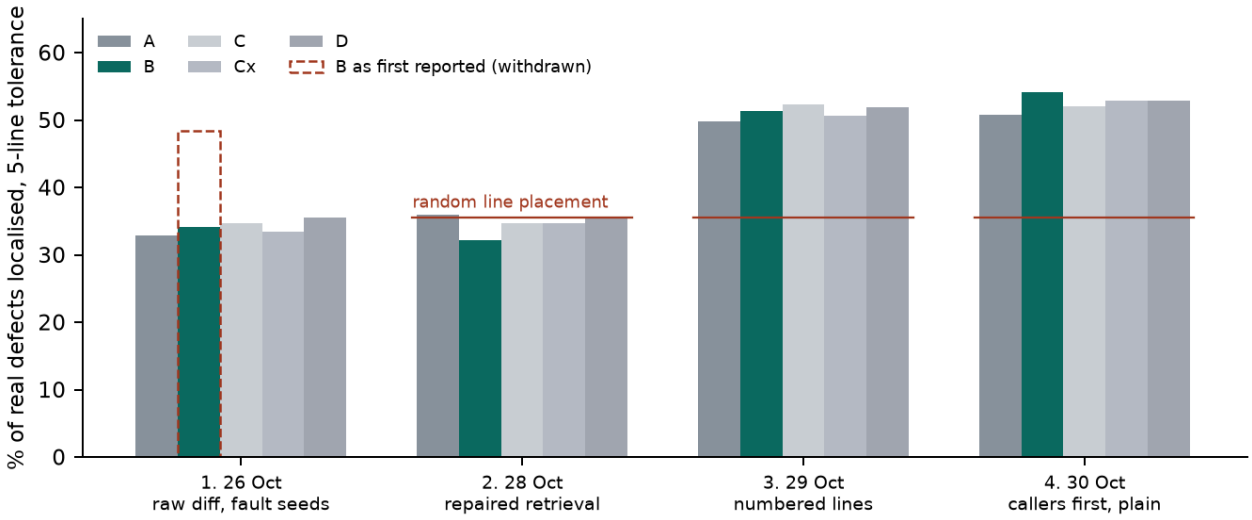


Figure 2. c-CRAB real defects, gpt-4o-mini, 5-line tolerance, the four runs side by side. Run 1's graph condition as first reported (dashed) carried the line span of the defect's function; its solid bar is the span-removed control. Runs 1 and 2 sit at the rate of random line placement; numbering the diff lines in runs 3 and 4 lifts every condition by fifteen points, and no condition is above the diff alone on the full defect set.

The model throughout is gpt-4o-mini, the tolerance is five lines, and the defect set is the 246 defects of 3.6 whose diff fits the pipeline, in 195 pull requests from 63 repositories; each run is analysed on the defects answered in all of its conditions. Four runs were made: the first on 26 October 2026, withdrawn; the other three pre-registered in `paper/PLAN.md` (§10, its amendment, and §11), each run once and reported whatever it gave, and the last declared the final one before it started. Every run compares a diff-only condition A against four retrieval conditions at one budget: B, a walk over the dependency graph; C, random entities of the same type mix as B's walk, from any file; Cx, random entities from files other than the changed ones; D, lexical retrieval on the seed names. Two features are common to all four and are disclosed because the tables do not show them: the prompt for the retrieval conditions carries one sentence saying a retrieval tool is available and the diff-only prompt does not, though no tool is callable in any condition and the context is pre-computed; and a hit is any finding within five lines of the defect, so a condition under which the model reports more findings has more chances, which is why every rate from the second run on is read against a random-placement baseline.

Run 1, withdrawn. The first run seeded every retrieval condition from the function whose line range covers the ground-truth defect line, so retrieval began at the fault location, which a reviewer would not know; and its graph tool alone returned that seed with its `line_range` and `metadata`, while the random and lexical tools returned the seed by name, type and path only. The graph condition alone was therefore told the span of the function containing the defect. An external reviewer found this on 27 October 2026. The control is the graph condition with the seed reduced to the entry the other conditions carry, on the same defects with the same model, budget (1,500 tokens) and tolerance (`experiments/run.py --conditions B --strip-seed`, `results/ccrab-Bs-gpt4omini/`):

Run 1	Localised, 223 paired	at zero tolerance
A diff only	33.2%	2.5%
Cx random nodes, other files only, same budget	33.2%	
B′ graph retrieval, seed span removed	**34.1%**	2.9%
B graph retrieval, as first reported (withdrawn)	48.4%	9.4%

Comparison	Difference	95% interval, repositories resampled
graph as reported − graph, span removed	+14.3	[+8.8, +19.8]

graph, span removed − diff only	+0.9	[−3.1, +5.1]
graph, span removed − random, other files	+0.9	[−2.9, +4.9]

The fourteen-point gain was the span. Every figure from the graph condition in `results/ccrab-today/B-outcomes.jsonl` is withdrawn (`results/WITHDRAWN-ccrab-graph.md`); the file is kept so the withdrawal can be checked. The other conditions of that run, diff only 32.9%, random nodes 34.7%, random from other files 33.5% and lexical 35.6% on 219 paired defects, stand as what they are; the next run shows that they are not a model result either.

Run 2: the repair, and a result at chance. The repair of 27 October changed three things for every retrieval condition. The graph builder's caller edges were rebuilt and validated against `jedi` on six repositories outside the task set: pooled recall 0.40 before and 0.92 after, precision 0.89 and 0.85 (`results/graph-validation/`). The seeds became every function or class overlapping a changed hunk, read from the new-file side of the diff and never from the ground truth. And the payload became the entity's source, cut at 40 lines, with no line range, metadata, depth or score, at 3,000 tokens for every retrieval condition. The pre-registered run of 28 October (`results/ccrab-v2/`, 239 paired defects, $0.82) gave A 36.0%, B 32.2%, C 34.7%, Cx 34.7%, D 35.6%; B − Cx −2.5 [−6.0, +1.0]; nothing excludes zero. Two checks on its records, both made before any further model call, say why the numbers sit where they do. A chance baseline places the same number of findings as the model on uniformly random lines of the diff's hunks and scores them with the same matcher (`experiments/chance_ccrab.py`, 500 draws per task): it gives 36.0% for A against the model's 36.0% and 34.9% for B against 32.2%, and at the exact line every condition is below chance, two of them with intervals excluding zero. And of the diff-only findings whose message quotes a token present in the diff, 459 of 1,014, 8.9% report the line that holds the token and 39.4% are more than five lines off (`experiments/line_accuracy.py`). The model was given a raw unified diff with no per-line numbers, had to count from each hunk header, and mostly did not; the ground truth is not at fault, since 237 of the 246 defect lines lie inside a hunk of their file. The generated benchmark is unaffected, because every one of its 1,103 hunks is one line on the new-file side. Run 2 measures the harness, and so, in retrospect, does every rate of run 1.

Run 3: numbered lines. The amendment of 29 October prefixed every context and added line of the diff with its new-file line number, identically for every condition, said so in one sentence of the output contract, and changed nothing else (`results/ccrab-numbered/`, 243 paired, $0.87). Quoted-token line accuracy went from 8.9% to 71.2% exact and from 39.4% to 3.3% more than five lines off. Every condition rose to about half: A 49.8%, B 51.4%, C 52.3%, Cx 50.6%, D 51.9%, each 15 to 17 points above its random-placement baseline with an interval excluding zero, and at the exact line 23 to 24% against 5% chance. B − Cx +0.8 [−1.3, +3.3]; B − A +1.6 [−1.1, +4.5]; at the exact line B − Cx +2.1 [−0.9, +5.1]; no paired difference excludes zero.

Run 4: the final run, with a primary comparison fixed in advance. An audit of run 3 found four weaknesses in how the context reached the model, none specific to one condition: entities arrived as a JSON envelope; the delivered ids were not logged; the graph walk returned test files and ranked neighbours breadth-first; and a class was seeded beside its own changed methods. §11 of the plan, committed before the run, rendered every entity as a plain code block under a path-and-name header, logged the delivered ids, excluded test files from what B, C and Cx draw so that B and Cx differ only in graph structure and ranking, ranked direct callers of the changed functions first, then callees, then depth two, and dropped class seeds whose methods were already seeds. It also fixed the primary comparison before the run: B − Cx on source-file defects whose reference comment is strict outside-diff under the classification of 4.2, 77 ids committed in advance, 64 of them paired, in 34 repositories. The plan discloses that this subset had been seen in exploratory analysis before it was chosen: on run 3's records it read B 54.7% against Cx 51.6%, and on run 2's, B 40.6% against Cx 42.2%. Run 4 (`results/ccrab-final/`, 244 paired, $0.76) gives:

Run 4, all 244 paired defects	localised, tol 5	random placement	exact line	chance

A diff only	50.8%	35.3%	23.4%	4.8%
B graph, callers first	54.1%	35.4%	24.6%	4.9%
C random, any file	52.0%	34.9%	24.2%	4.8%
Cx random, other files	52.9%	35.1%	24.6%	4.6%
D lexical	52.9%	35.3%	25.0%	4.8%

Comparison, repositories resampled	**all 244 defects**	**pre-registered subset, 64 outside-diff defects**
B − Cx (the primary, on the subset)	+1.2 [−2.2, +5.0]	**+12.5 [+5.0, +20.8]**
B − A	+3.3 [−0.9, +7.2]	**+12.5 [+3.6, +22.4]**
Cx − A	+2.0 [−1.7, +5.2]	0.0 [−4.5, +4.8]
D − A	+2.0 [−1.8, +5.4]	+3.1 [0.0, +8.1]
B − Cx at the exact line	+0.4 [−2.1, +3.4]	+3.1 [−3.2, +9.0]

On all defects nothing excludes zero. On the subset B localises 41 of 64 against 33 for Cx and 33 for A; the eight defects that separate B from Cx are all ones B finds and Cx does not, seven repositories favour B and none the other way, and the task-resampled interval, [+4.7, +20.3], agrees. At the exact line the gap is +3.1 and spans zero, so the extra hits land within the tolerance rather than on the line. Evidence recall, the share of defects whose comment names a file or identifier that the delivered context contains, is B 61.9%, D 60.4%, C 50.6% and Cx 50.0% on all defects, and on the 85 whose comment names something outside the diff, D 22.6%, B 18.8%, Cx 14.1%, C 11.8% (`evidence-recall.txt`). The subset comparison passed the test the plan set for it, and we still report it as a lead rather than a result: it is eight defects on one model, in a subset chosen after two readings of it, one +3.1 and one −1.6, and neither the exact-line rate nor evidence recall singles B out.

What stands. Across four runs and three harness repairs (Figure 2):

Run, paired defects	**A**	**B**	**C**	**Cx**	**D**	**B − Cx**
1, 26 Oct: fault-location seeds, 1,500 tokens, raw diff (219; B is the span-removed control, 223)	32.9	34.1	34.7	33.5	35.6	+0.9 [−2.9, +4.9]
2, 28 Oct: repaired graph, diff seeds, code payloads, 3,000 tokens (239)	36.0	32.2	34.7	34.7	35.6	−2.5 [−6.0, +1.0]
3, 29 Oct: numbered diff lines (243)	49.8	51.4	52.3	50.6	51.9	+0.8 [−1.3, +3.3]
4, 30 Oct: plain rendering, callers first, no test files (244)	50.8	54.1	52.0	52.9	52.9	+1.2 [−2.2, +5.0]

Random line placement, computed for runs 2 to 4, is 35 to 36% in every condition. No comparison on the full defect set excludes zero in any run. Runs 1 and 2 sit at chance for every condition and measure nothing about context; runs 3 and 4 sit fifteen points above it and still show no condition above the diff alone on all defects. The one interval that excludes zero is the pre-registered subset of run 4, where the reference comments say the evidence lies outside the diff, and it is a lead for a study with a second model and a subset fixed before any look at the data. No c-CRAB iteration follows run 4.

4.6 The search-trigger test

One model (gemini-2.5-flash), the same two tools, the same repository and the same diff; one sentence of framing varies. Across 40 paired tasks a bare tool-equipped model searched in 0 of 40 under the review prompt and in 40 of 40 when told the defect depended on another file, reaching the caller in 20%. The result is scoped to a bare model: harnessed products explore unprompted [32].

4.7 The withdrawn and stopped runs

The held-constant experiment of 3.5 was run in full on the second benchmark: four base runs across three model families and two serialisations, two twin runs, a hidden-evidence and a noise variant, and an authoritative-callers variant, each judged by two models. Every number from those runs is withdrawn with the benchmark, for the five instrument defects listed in 3.2; the raw records, the analyses and the judge labels are kept under `results/` with a notice (`results/WITHDRAWN-v2-structure.md`) so that the withdrawal can be checked, and so that a reader can see what the effects looked like on a leaking instrument. On the third benchmark one base run was started and stopped under the pre-set rule at 201 defects; its nine arms are kept whole in `results/v3-ceiling/` and its arm-level numbers are not reported as results, because a comparison between arms on a benchmark where the diff alone scores 95.5% is a comparison of what the model does with the other 4.5%.

One number from that run is reported, as a signal and not as a result of the experiment, because it is judge-free, because it answers a question the twins also answer, and because it does not depend on detection: precision, whether the mutated line is the only line flagged. On the 200 defects paired across all nine arms (`results/v3-ceiling/analysis-precise-partial.txt`; one model, one serialisation, stopped run):

Comparison on precision	Difference	95% interval, repositories resampled
real caller shown (evidence) − diff only	−14.0	[−20.7, −7.9]
real caller + attributed relation − diff only	−11.5	[−17.6, −5.6]
foreign code, same length − diff only	+6.0	[+1.7, +10.1]
header only − diff only	−3.0	[−7.5, +1.1]
bare link − scrambled control	0.0	[−3.2, +3.0]
attributed relation − scrambled control	+2.5	[−0.9, +6.2]

Showing the model the caller that depends on the change makes it flag more of the harmless lines, by fourteen points against the diff alone, while foreign code of the same length makes it flag slightly fewer. Stating the link between the snippets recovers none of the loss: the bare link against the scrambled control is 0.0 with an interval of three points either side. The withdrawn second-version runs had reported a ten-to-fourteen-point precision gain for the bare link on all four runs; on the repaired benchmark, where labels no longer carry line numbers, that gain is absent on the one run made. The second version's structure effect on precision is therefore best read as the line-number leak, and the twins in 4.3 are the direct measurement of what the caller does to false alarms.

5. Discussion

What the diagnostics say about benchmarks. A benchmark can be carefully built and still be unable to measure context. Three versions of ours, the last with every known leak closed and audited, give the diff alone 93%, 97.8% and 95.5% of the defect lines under a prompt that presumes a defect, and 30% under one that does not. A study of interprocedural context for vulnerability detection reports that adding callers and callees does not raise accuracy for the models it tests, and lowers it for several [41], and ContextCRBench finds that added code lowers line-level localisation [42]. None of these benchmarks is wrong about what it contains; each is silent about context because the diff already answers the question. The partial-input baseline is a one-line check, and the lesson of the third version is that it has to be run on the repaired instrument, not assumed to improve with the repairs. The lesson of the real defects is that it has to be read against a chance baseline too: on the raw diff every condition's 36% was the rate of random line placement, and the harness, not the model, had set it.

Why the ceiling survived the repair. The second version leaked in five identifiable ways, and closing them moved the ceiling by two points. What remains is not a leak. The model flags 2.7 of the 4 changed lines per task; it does not pick the defect out, it declines to leave much out, and a semantic edit among cosmetic ones is always in the set. Two things follow. A mutation benchmark that wants detection to carry information needs distractors that

are themselves semantic and plausible, several changed lines any of which could be the defect, or more than one defect per diff; cosmetic distractors test whether the model can spot the one edit that does something, and it can. And the metric that still moves on such a benchmark is precision, whether the model also flags the harmless lines, which is a property of restraint rather than of understanding. We did not run the arms on the third version to measure it, because the pre-set rule did not allow a partial run to become a result. The neutral-prompt run in 4.3 adds a third thing: the ceiling is also the prompt. Told that the diff has a defect, the model names the semantic edit 95% of the time; allowed to say nothing, it names it 30% of the time, and the arms then separate, but by flagging rather than by discriminating, since every gain on the defects is matched by a loss on the twins and balanced accuracy does not move.

Three questions, and what happened to each. The held-constant experiment was built to answer three questions the real-defect result cannot, because real pull requests cannot be controlled: whether the model understands the defect or only points at its line; whether context causes false alarms; and whether it is the caller's code or the description of the link that helps. The first needed the model judges, and nothing from them is reported, so it is not measured. The second is answered by the twins under a neutral prompt: the caller that shows a change to be safe costs 27 points of restraint on gpt-4o-mini and 2 on deepseek-v3.2, and each statement about the dependency costs more, down to one in nine on the first model and four in five on the second with the relation named. The third cannot be answered for detection on a benchmark where the diff alone finds the line 95% of the time under the presuming prompt, and both attempts are withdrawn; under the neutral prompt the code and the description are separable and both move the model, the code by 22 points of detection and 27 of restraint, the bare link by 11 more and 15 more, and the balanced accuracy of every arm that carries evidence is within noise of, or below, the diff alone. What moves is the willingness to flag, on defects and safe changes alike. What the experiment answered instead is a question it was not designed for: whether a planted-defect benchmark can measure context at all. Two of the three questions are open, and the next step is a benchmark on which the diff alone cannot find the defect.

Where context can be measured, and what was found there. The real defects are the set with headroom, and on them this project made four runs. The first reported a retrieval result it now withdraws: graph retrieval appeared to localise 48% against 33 to 36% for every control, and the gain was a line span the graph condition alone had been given. The second, with the retrieval repaired, put every condition at the rate of random line placement, and the check that showed why, fewer than one quoted line in ten reported on the line that holds it, is one any benchmark that scores by line should run: a raw unified diff asks the model to count from hunk headers, and the harness, not the model, had set a ceiling of 36%. With the lines numbered, every condition sits at about half and fifteen points above chance, and on the full defect set nothing separates graph retrieval from random code of the same size, in two runs. The anchor-versus-evidence measurement says why there is room: a third to a half of the reference comments justify themselves with code the diff does not show. On exactly those defects, the pre-registered subset of the final run, the callers-first walk is twelve points above random code from other files and above the diff alone, with an interval that excludes zero; it was chosen with earlier readings of +3 and −2 in view, it is eight defects on one model, and we call it a lead. Two readings of the full-set null stay open: that the model cannot use a caller it is handed, which the twins under the neutral prompt make plausible, since there the caller is read as an accusation rather than as evidence; or that for most real defects the justification is not where a 3,000-token neighbourhood of the changed functions looks. The subset lead favours the second reading, and a second model would decide it.

How this sits with SWE-PRBench and AACR-Bench. SWE-PRBench reports that eight frontier models get worse as same-file context is appended to the diff, and AACR-Bench reports that moving from diff to file to repository granularity changes performance in a direction that depends on model and language [12, 2]. Both vary the *amount* of context. We hold the amount fixed at 3,000 tokens and vary the *selection*, and on the full defect set our data reproduce their null: with the diff lines numbered, the diff alone localises 50.8%, random code from other files 52.9%, lexical retrieval 52.9% and the callers-first graph walk 54.1%, and no difference excludes zero. On

real defects, for this model and budget, no selection method we tried moves the number on all defects, which is their finding restated with the selection controlled; the one movement is on the 64 defects whose reference comment cites code outside the diff, and it is a lead. The twins add the part their designs cannot see: when the dependent code is handed to the model on a safe change, it raises false alarms, and a statement of the dependency raises them further. The comparison has limits: their models are at the frontier and ours are not, and they score whole pull requests where we score located defects.

The twins. Under the presuming prompt no arm on either model clears more than 11% of the changes every sampled caller tolerates, and on gpt-4o-mini none clears more than 4%; under a neutral prompt the same model clears 82% from the diff and 11% once the relation is named, and deepseek-v3.2 goes from 89% to 79% in the same order, so the prompt had hidden an effect whose direction both models share and whose size is the model's. The design's largest judge-free finding is that structure stated in the prompt drives flagging: on the twins it drives false alarms, on the defects it drives detection by the same steps, and balanced accuracy over the two is 56% for the diff alone, no better for the caller or the bare link, and ten points worse for the attributed link. Foreign code of the same length does none of this. Under the plain prompt the extra flagging is partly defensible, since the prompt presumes a defect and the function is public; under the neutral prompt it is not, and it is the direction a context method has to reverse before it can claim to help a reviewer rather than to help a detector. A wrong link costs as much as a right one on gpt-4o-mini, on the twins and on the defects, so the model is not reading the relation and finding it violated; it is reading that a relation exists and treating that as the defect. The discrimination task is where a context method would have to show its value, and the benchmark now contains it with twins that are safe across every sampled file, under both prompts. The signal-detection reading of 4.3 says the same in two numbers: d' never rises above the diff's 0.37 in any arm that carries evidence, and c falls from $+0.71$ to -1.07. This is the pattern that "Words Speak Louder Than Code" [25] reports for vulnerability detection, where context that leaves the code unchanged, an author's name, a framing, a prior verdict, moves eight models' verdicts from safe to vulnerable without the model identifying a vulnerability. Structural dependency context behaves as one more such cue in code review: the caller and the stated relation are read as reasons to flag, on defect and safe change alike.

What a generated benchmark owes its reader. Two numbers that a mutation benchmark can always report and that ours did not report at first: the diff-only rate, and the fraction of defects an existing test catches. The second is 38% of those we could run, which says the syntax-tree verifier accepts many mutations the repository's own tests would not notice. Those mutations are not harmless, but a reader is entitled to know that most of the benchmark's ground truth is verified by static reasoning rather than by execution, and a benchmark that cannot be installed for a quarter of its repositories should say that too.

What this does not show. Nothing here speaks to dependencies a static call graph cannot express, to languages other than Python, to models at the frontier, or to tool-using agents, which explore unprompted and for which the search-trigger result does not apply. Nothing here says whether dependency metadata can help a model find a defect without also making it flag safe changes; on the one benchmark and model where both were measured under the same prompt, the two rose together and balanced accuracy did not. What it says is that on a safe change the metadata hurts, that on generated defects it raises detection only as far as it raises false alarms, and that on real defects no retrieval tried here helps.

What the record shows. Eight claims were made and withdrawn during this work, each on the project's own evidence, and Section 6 lists them. The last three are the held-constant experiment on each generated benchmark and the retrieval result on real defects. The design was pre-registered, the controls were matched, the judges were validated and the analysis resampled repositories; none of that protected it from an instrument that gave the answer away, twice on generated defects and once on real ones, each time through a line number or a line span that only one condition received. The diagnostics in this paper are what we would run first next time, and the control that withdrew the last claim cost less than a dollar and an afternoon.

6. Limitations and threats to validity

The defects are generated, and the generator conditions on what it later evaluates. A task exists only when the caller's syntax tree demonstrates the dependency the mutation breaks. That makes the evidence exact, and it also means every defect is one for which a two-file evidence set exists by construction. Real cross-file defects frequently arrive through channels a static call graph does not represent: dynamic dispatch, event listeners, dependency injection, framework lifecycle hooks, reflection, schema mismatches, configuration. Nothing here speaks to those. The 96 mined regression candidates we release are the external check we have not yet run.

Most generated defects are not verified by the repositories' own tests. Section 4.4 reports the funnel: of 86 attempted, 17 are test-verified, 28 are not caught by the selected tests, and 41 could not be run to a verdict. A fifth of those attempted, and 38% of those with a verdict, is lower than we would like. It says the syntax-tree verifier accepts many mutations whose breakage no existing test observes; it does not say those mutations are harmless. On the third version, 32 of 277 are test-verified, 27 are not caught, and 218 could not be run to a verdict, most because the repository did not install in the sandbox; the executable subset is listed and a reader may restrict to it.

The ceiling is measured on one model. The diff-only rate is reported for gpt-4o-mini on every version. The second version's runs, withdrawn, put llama-3.3-70b and deepseek-v3.2 at the same ceiling, so we do not expect a cheaper model to lower it, but on the third version it is one model.

The precision signal comes from a stopped run. The third version's base run was halted at 201 of 277 defects because the diff-only rate exceeded the pre-set threshold, and the rule said to report only the ceiling. We report one more number from it, the precision comparison in 4.7, because it is judge-free and because the twins measure the same thing directly; it is one model, one serialisation and 200 paired defects, it was not pre-registered on this version, and a reader should weigh it as a signal that the twins either confirm or do not.

Python only. The generator, the import resolution and the verifiers are written against Python's `ast`. The project's graph builder parses four languages; the benchmark exercises one.

The pools are samples. Each repository contributes up to three hundred production files chosen at random, not the whole repository, so a caller that exists in the repository may be absent from the pool and a function that is called in many places may be represented by one. Twin safety is checked against the pool, so a caller outside the sampled files could depend on a change the pool calls safe. Yield per repository varies from one to eight for this reason.

The reviewer prompt presumes a defect. One sentence in the system prompt tells the model that humans found real problems in the diff and to assume there is something to find. It is identical across arms, so no comparison between arms is confounded by it, but every absolute rate is: under it the twin clear rates sit at a floor on gpt-4o-mini, and the diff-only ceiling is that of a model told to flag. The neutral variant was run on the twins and on the third version's defects for one model: it raised the diff-only clear rate from 2.5% to 82% and lowered the diff-only detection rate from 95.5% to 30%, which is the size of the effect. The ceiling in 4.1 and the precision signal in 4.7 are under the presuming prompt; the numbers to compare with other work are the neutral-prompt ones in 4.3. The c-CRAB runs in 4.5 are under the presuming prompt throughout, where every diff does carry a confirmed defect.

The twins may be unfair to a prudent reviewer. The mutated functions are public. A twin is safe for every caller in the pool, and a change to a public default or exception type can still break code outside the repository; a reviewer that flags it anyway is applying a sound rule. Twins built on private functions, where the pool is the whole truth, would settle it and are the right next construction. Twins exist for two of the five mutation kinds, most of them default flips, and on gpt-4o-mini the clear rate is at a floor where no arm comparison can be read.

The retrieval runs changed between iterations, and the first started from the fault location. The withdrawn run and its control seeded every retrieval condition from the function covering the ground-truth line, which a reviewer would not know, and the graph condition alone received that function's line range and metadata; Section 4.5 reports the control that measured what the span was worth, fourteen points, the whole of the reported gain. The three later runs seed from the diff's changed functions, deliver code rather than names, and were each pre-

registered; two of them changed the harness (numbered diff lines, then plain rendering with test-free, callers-first retrieval), so the four runs are four configurations, reported side by side and not pooled. The null on the full defect set is one model, one budget and one window size; the lead on the outside-diff subset is one model, 64 defects and a subset chosen with earlier runs in view. A frontier model, a larger budget or a wider window could move either, and a reader should take the null as a bound for this configuration, not as a demonstration that retrieval cannot help.

The graph keys methods without their class. The builder keys a method as file and function name with no class, so two classes in one file with a method of the same name share a node and their callers merge. In the validation against `jedi` on six repositories outside the task set (`results/graph-validation/`), 69 of the repaired builder's 84 false-positive caller edges were on methods, consistent with this collision; the repair raised pooled recall from 0.40 to 0.92 and lowered precision from 0.89 to 0.85. The collision touches condition B only, was stated in advance of the three later runs and not fixed for any of them, and can only add wrong neighbours to B's payload, so it works against B.

Statistical power. Pairing across arms and the McNemar test give the design its power; the repository-clustered interval is wider than the task-clustered one because tasks within a repository share files. A null with an interval of several points in either direction is a bound, not a demonstration of equivalence.

Scope of the agent result. The search-trigger measurement is on a bare model with two tools and no harness. Harnessed products explore unprompted [32]; the result says nothing about them.

The history of this study. Eight claims were made and withdrawn during this work, each on the project's own evidence, and this paragraph is the only place the list appears: that benchmarks cannot contain cross-file defects; that a fault-location oracle was an evidence oracle; that random context beat graph retrieval; that retrieval relevance was undetectable; that the retrievers were comparable in what they retrieved; that the first generated benchmark measured what it claimed; that the second did; and that graph retrieval localised fifteen points more real defects than any control, which was the line span of the defect's function, given to that condition alone (`results/WITHDRAWN-ccrab-graph.md`). The first generated benchmark's distractors did not parse in 145 of 204 tasks, its evidence demonstrated the dependency in roughly a third of tasks, and its diff-only arm found the defect line 93% of the time. The second's labels carried the mutated line's number, its resolver accepted a same-name import from any module in 36 of 229 tasks, 25 of its 150 twins had a dependent call in a shown file, 91 of its 650 distractors changed the syntax tree, and an unparseable reply was scored as correctly silent; an external review on 26 October 2026 found these, and `results/WITHDRAWN-v2-structure.md` records them. Every number from both is withdrawn and the files are kept with a notice. The third benchmark was generated after those faults were found, with a test for each, and its diff-only rate under both prompts, with the twins under both prompts and the four-run record on real defects, is the result of this paper.

7. Artifact

Everything needed to regenerate the benchmark and the twins, audit them, re-run every diagnostic, and recompute every number is in the repository, with the frozen outputs beside the code.

Benchmark generator. `experiments/crossfile.py` holds the five mutation kinds, the import resolver with re-export following, the per-kind verifiers that inspect the caller's syntax tree, the repository-wide dependent-call scan used for twins, and the syntax-tree-neutral distractors with their seeded placement. `experiments/rebuild_crossfile.py --from-pools` regenerates `data/crossfile-v3.jsonl` from the cached repository pools (`experiments/_fetch_sources.py` fetches them at the pinned commits), `experiments/build_twins.py` regenerates `data/crossfile-v3-twins.jsonl`, and `experiments/build_chains.py` the six two-hop chains. The funnel is printed, not hidden.

Audit. `experiments/audit_v2.py` re-checks any released defect or twin file with the generator's own resolver and tree comparison and writes `results/audit/<label>-recheck.txt`; the third version's audit and the second

version's, which found the leaks, are both kept.

Diagnostics. `experiments/run_structure.py` runs the diff-only arm and the others, records the parsed findings, the exact prompt length and the cost of every call, and resumes; `experiments/evidence_scope.py` classifies c-CRAB comments by anchor and evidence scope with the reason recorded for every decision; `experiments/verify_by_tests.py` runs each repository's own tests before and after each mutation and reports the funnel; `experiments/run_search_trigger.py` is the search-trigger test.

Held-constant experiment. `experiments/evidence.py` builds the evidence set, the scrambled control, the typed and attributed relations, the corruption, the length-matched foreign-repository arm and the hidden-evidence window; `experiments/judge_mechanism.py` is the two-judge scoring and `experiments/analyse_structure.py` the paired analysis with repository-clustered and task-clustered intervals, McNemar's exact test and a Holm-adjusted family. They are kept for a benchmark that passes the gate.

Real defects. `experiments/run.py` runs the retrieval conditions on c-CRAB, `--strip-seed` is the control that withdrew the first run, the launcher kept beside each run directory reproduces each of the three later runs, `experiments/analyse_ccrab.py` and `experiments/analyse_ccrab_subset.py` recompute Section 4.5 with a fixed seed, `experiments/chance_ccrab.py` is the random-placement baseline, `experiments/line_accuracy.py` the quoted-token line check, `experiments/evidence_recall.py` the evidence-recall measure and `experiments/signal_detection.py` the d′ and c of 4.3; `experiments/mine_regressions.py` mines the 96 real cross-file regression candidates released in `data/regressions-candidates.jsonl`.

Reproduction. `scripts/reproduce.py` recomputes every analysis file in `results/` from the raw records beside it and compares line by line, offline, in about two minutes; `REPRODUCE.md` maps each number in Section 4 to its input file and command.

Frozen results. `results/` holds one directory per run with the raw per-call records and the analysis output. The second-version runs carry `WITHDRAWN-v2-structure.md`, the graph condition on c-CRAB carries `WITHDRAWN-ccrab-graph.md` beside the control that withdrew it, and the three pre-registered runs that followed are `results/ccrab-v2/`, `results/ccrab-numbered/` and `results/ccrab-final/`, each with its launcher, cost log, analyses and a 50-finding hand-label sample; the stopped third-version run is in `results/v3-ceiling/` with its stop rule and the spend; the test-verification funnel is in `results/test-verified/`.

Tests. More than 600 unit tests, including one for each fault found in each withdrawn benchmark: a distractor that changes the syntax tree is rejected, a same-name import from another module is not evidence, a twin with a dependent call anywhere in the pool is rejected, a label never carries a line number, an unparseable reply is scored as missing, a scrambled control contains no true token, a corrupted relation is never a self-loop.

Not included. Model outputs were obtained through OpenRouter at the prices recorded in each run's records; model versions change, and exact replication of a number is not guaranteed. The 96 regression candidates are unverified and are released as candidates.

References

[1] Xingliang Wang, Baoyi Wang, Chen Zhi, Junxiao Han, Xinkui Zhao, Jianwei Yin et al. GRACE: Graph-Guided Repository-Aware Code Completion through Hierarchical Code Fusion. 2025. ICSE 2026. arXiv:2509.05980

[2] Lei Zhang, Yongda Yu, Minghui Yu, Xinxin Guo, Zhengqi Zhuang, Guoping Rong et al. AACR-Bench: Evaluating Automatic Code Review with Holistic Repository-Level Context. 2026. arXiv:2601.19494

[3] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, Noah A. Smith. Annotation Artifacts in Natural Language Inference Data. 2018. NAACL 2018. arXiv:1803.02324

[4] Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, Benjamin Van Durme. Hypothesis Only Baselines in Natural Language Inference. 2018. *SEM 2018. arXiv:1805.01042

[5] Shi Feng, Eric Wallace, Jordan Boyd-Graber. Misleading Failures of Partial-input Baselines. 2019. ACL 2019. arXiv:1905.05778

[6] Yuntong Zhang, Zhiyuan Pan, Imam Nur Bani Yusuf, Haifeng Ruan, Ridwan Shariffdeen, Abhik Roychoudhury. Code Review Agent Benchmark. 2026. arXiv:2603.23448

[7] Matt Gardner, Yoav Artzi, Victoria Basmova, Jonathan Berant, Ben Bogin, Sihao Chen et al. Evaluating Models' Local Decision Boundaries via Contrast Sets. 2020. Findings of EMNLP 2020. arXiv:2004.02709

[8] Divyansh Kaushik, Eduard Hovy, Zachary C. Lipton. Learning the Difference that Makes a Difference with Counterfactually-Augmented Data. 2020. ICLR 2020. arXiv:1909.12434

[9] Zhengran Zeng, Ruikai Shi, Keke Han, Yixin Li, Kaicheng Sun, Yidong Wang et al. SWR-Bench: Assessing LLM Performance in Real-World Code Review Comment Generation. 2025. FSE 2026. arXiv:2509.01494

[10] Hanyang Guo, Xunjin Zheng, Zihan Liao, Hang Yu, Peng DI, Ziyin Zhang et al. CodeFuse-CR-Bench: A Comprehensiveness-aware Benchmark for End-to-End Code Review Evaluation in Python Projects. 2025. arXiv:2509.14856

[11] Dewu Zheng, Yanlin Wang, Xiwen Wang, Kefeng Duan, Hongyu Zhang, Xilin Liu et al. From Static to Dynamic: Benchmarking Real-World Code Review with MCR-Bench. 2026. ISSTA 2026. arXiv:2608.27442

[12] Deepak Kumar. SWE-PRBench: Benchmarking AI Code Review Quality Against Pull Request Feedback. 2026. arXiv:2603.26130

[13] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia et al. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. 2024. ICLR 2025. arXiv:2410.14684

[14] Zhongxin Liu, Zhonghao Jiang, Zhifan Ye, Haoye Wang, Jiakun Liu, Xiaoxue Ren. Effective and Efficient Context Retrieval via Partial Dependency Graph for Repository-Level Code Generation. 2026. ASE 2026. arXiv:2608.01927

[15] Yuntong Hu, Tongli Su, Liang Zhao, Bowen Zhu, Hasibul Haque. LARGER: Lexically Anchored Repository Graph Exploration and Retrieval. 2026. arXiv:2605.16352

[16] Baoyi Wang, Xingliang Wang, Guochang Li, Chen Zhi, Junxiao Han, Xinkui Zhao et al. Better Call Grep: Evaluating and Improving Grep-Like Lexical Retrieval for Repository-Level Code Completion. 2026. ISSTA 2026. arXiv:2601.23254

[17] Bowen Qin, Yi Xie. Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents. 2026. arXiv:2607.24882

[18] Haotian Xu, Yuning You, Tengfei Ma. When Structure Doesn't Help: LLMs Do Not Read Text-Attributed Graphs as Effectively as We Expected. 2025. LoG 2025. arXiv:2511.16767

[19] Ben Finkelshtein, Silviu Cucerzan, Sujay Kumar Jauhar, Ryen White. Actions Speak Louder than Prompts: A Large-Scale Study of LLMs for Graph Inference. 2025. ICLR 2026. arXiv:2509.18487

[20] Daniel Herbst, Lea Karbevska, Divyanshu Kumar, Akanksha Ahuja, Fatemeh Gholamzadeh Nasrabadi, Fabrizio Frasca. Lost in Serialization: Invariance and Generalization of LLM Graph Reasoners. 2025. ICML 2026 Workshop on Graph Foundation Models. arXiv:2511.10234

[21] Donald Loveland, Puja Trivedi, Ari Weinstein, Edward W Huang, Danai Koutra. Formalizing and Mitigating Structural Distortion in LLM Attention for Graph Reasoning. 2026. KDD 2026. arXiv:2606.15633

[22] Xu Chu, Hanlin Xue, Zhijie Tan, Bingce Wang, Tong Mo, Weiping Li. GraphSOS: Graph Sampling and Order Selection to Help LLMs Understand Graphs Better. 2025. arXiv:2501.14427

[23] Yuyao Ge, Shenghua Liu, Baolong Bi, Yiwei Wang, Lingrui Mei, Wenjie Feng et al. Can Graph Descriptive Order Affect Solving Graph Problems with LLMs?. 2024. ACL 2025. arXiv:2402.07140

[24] Andrew Stoltman, Johnathan Tang, Haipeng Cai. Representation Matters: An Empirical Study of Program Representations for LLM Vulnerability Reasoning. 2026. arXiv:2606.25356

[25] Asif Shahriar, Hongyu Cai, Hadjer Benkraouda, Gang Wang, Z. Berkay Celik. Words Speak Louder Than Code: Investigating Cognitive Heuristics in LLM-Based Code Vulnerability Detection. 2026. IEEE S&P 2027. arXiv:2606.30587

[26] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan et al. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. 2025. Findings of EMNLP 2025. arXiv:2510.05381

[27] Yanlin Wang, Suiquan Wang, Yanli Wang, Bowen Zhang, Daya Guo, Jiachi Chen et al. RepoReasoner: Evaluating Repository-Level Code Reasoning Ability of Long-Context Language Models. 2026. FSE 2026. arXiv:2607.25996

[28] Haojun Weng, Qianqian Yang, Hao Fu, Haobin Pan, Xinwei Lv. When Retrieval Hurts Code Completion: A Diagnostic Study of Stale Repository Context. 2026. Information and Software Technology, 2026. arXiv:2605.14478

[29] Georgios Alexopoulos, Nikolaos Alexopoulos, Diomidis Spinellis, Dimitris Mitropoulos. Measuring and Exploiting Contextual Bias in LLM-Assisted Security Code Review. 2026. arXiv:2603.18740

[30] Scott Thornton. Can Adversarial Code Comments Fool AI Security Reviewers -- Large-Scale Empirical Study of Comment-Based Attacks and Defenses Against LLM Code Analysis. 2026. arXiv:2602.16741

[31] Tarakanath Paipuru. CodeCompass: Navigating the Navigation Paradox in Agentic Code Intelligence. 2026. arXiv:2602.20048

[32] Wachiraphan Charoenwet, Kla Tantithamthavorn, Patanamon Thongtanunam, Hong Yi Lin, Minwoo Jeong, Ming Wu. Agentic Code Review in the Terminal: A Trajectory-Level Analysis of Behavior, Cost, and Human-Alignment. 2026. arXiv:2607.16740

[33] Zhengfeng Li, Lei Zhang, Xianwei Wu, Zhengqi Zhuang, Yingjie Xu, Boge Wang et al. OpenCodeReview: Determinism over Non-Determinism for Cost-Effective Agent-Based Code Review. 2026. arXiv:2608.09290

[34] Kristen Pereira, Neelabh Sinha, Rajat Ghosh, Debojyoti Dutta. CR-Bench: Evaluating the Real-World Utility of AI Code Review Agents. 2026. ICLR 2026 workshop. arXiv:2603.11078

[35] Xuezhi Song, Yijian Wu, Bihuan Chen, Zhengjie Lu, Shuning Liu, Xin Peng. Characterizing Regression Bug-Inducing Changes and Improving LLM-Based Regression Bug Detection. 2026. ICSE 2026 (Research Track). github.com/FudanSELab/LLM4Reg

[36] Yunbo Lyu, Jieke Shi, Hong Jin Kang, Ratnadira Widyasari, Junda He, Yuqing Niu et al. AgentSZZ: Teaching the LLM Agent to Play Detective with Bug-Inducing Commits. 2026. arXiv:2604.02665

[37] Haolin Jin, Huaming Chen. Are LLMs Reliable Code Reviewers? Systematic Overcorrection in Requirement Conformance Judgement. 2026. arXiv:2603.00539

[38] Gokul Chandra Purnachandra Reddy, Aditya Lolla, Harsha Sanku. Articulate but Wrong: Self-Review Failures in LLM-Based Code Modernization. 2026. arXiv:2605.21537

[39] Feiyang Huang, Yuqiang Sun, Fan Zhang, Ziqi Yang, Han Liu, Yang Liu. Do Fine-Tuned LLMs Understand Vulnerabilities? An Investigation into the Semantic Trap. 2026. arXiv:2601.22655

[40] Zixiao Zhao, Amirreza Esmaeili, Fatemeh Fard. Bias in the Loop: Auditing LLM-as-a-Judge for Software Engineering. 2026. arXiv:2604.16790

[41] Kevin Lira, Baldoino Fonseca, Davy Baía, Márcio Ribeiro, Wesley K. G. Assunção. Vulnerability Detection with Interprocedural Context in Multiple Languages: Assessing Effectiveness and Cost of Modern LLMs. 2026. arXiv:2604.08417

[42] Ruida Hu, Xinchen Wang, Xin-Cheng Wen, Zhao Zhang, Bo Jiang, Pengfei Gao et al. Benchmarking LLMs for Fine-Grained Code Review with Enriched Context in Practice. 2025. arXiv:2511.07017